

Java Coursework 2026

AI-Assisted Information Management System

for Honor of Kings

Java OOP | Responsible AI Use | Git Process Evidence

Total Marks

20 marks

Submission

Java source code, documentation, AI evidence, Git history, and testing evidence

Core Idea

Students may use AI tools, but they must record prompts, show human judgment, preserve Git evidence, and explain Java design decisions.

Recommended Mode

Console application first; GUI or advanced features only after core requirements are stable.

Design Philosophy

This redesigned assignment treats AI use as an assessed academic practice rather than a hidden shortcut. Students must demonstrate both programming competence and AI literacy: planning, prompting, verification, debugging, reflection, and accountability.

Instructor-facing handout. Adapt course code, deadline, platform, and local academic integrity rules before release.

Contents

1	Pedagogical Rationale	3
2	Coursework Overview	3
2.1	Project Title	3
2.2	Task Summary	3
2.3	Learning Outcomes	4
3	Core Java Design Requirements	4
3.1	Required Classes	4
3.2	Required Java Concepts	4
4	Initial Dataset Requirements	5
5	Functional Requirements	5
5.1	Player Lookup	5
5.2	Team Overview	6
5.3	Hero Details	6
5.4	Equipment Statistics	6
5.5	Match History	6
5.6	Leaderboard	7
5.7	Data Management	7
5.8	Authentication	7
6	Required AI Usage and Evidence	7
6.1	Required AI Folder	8
6.2	Prompt Record: prompts.md	8
6.3	Multi-Agent Evidence: agent-log.md	9
6.4	Reflection: reflection.md	9
7	Required Git Evidence	10
7.1	Minimum Git Requirements	10
8	Required plan.md	11
8.1	Suggested Timeline	11
9	Source Code and Testing Requirements	11
9.1	Source Code Requirements	11
9.2	Testing Requirements	12

10 Extra Credit Features	13
10.1 Combat Simulation	13
10.2 Recommendation Engine	13
10.3 GUI	13
10.4 Data Persistence	13
10.5 Advanced AI Reflection	14
11 Submission Requirements	14
12 Academic Integrity and AI Policy	14
13 Grading Rubric	15
13.1 Grade Bands	16
14 Minimum Passing Checklist	16
Appendix A: Student Workflow	17
Appendix B: Prompt Quality Guide	17
Appendix C: Recommended README.md Structure	17
References	18

1 Pedagogical Rationale

This coursework redesign keeps the original **Honor of Kings information management system** theme and its Java OOP foundation, but adds a structured AI-use layer. The revised assignment is designed to assess two abilities at the same time:

- whether students understand Java programming, object-oriented modelling, collections, file I/O, exception handling, and testing;
- whether students can use AI tools responsibly, transparently, and critically in an academic programming task.

The design is informed by three Stanford-related teaching ideas. First, Stanford CS106B's AI coding component asks students to use an LLM and then report their model choice, prompts, integration process, debugging, and reflection [1]. Second, Stanford Teaching Commons recommends making AI policy explicit in course materials, including what is allowed, what must be disclosed, and how the policy supports academic integrity [2]. Third, Stanford SCALE's "Prompt Problems" framing shifts part of programming education toward constructing prompts, reading generated code, and evaluating correctness rather than only writing code from scratch [3].

Assessment Principle

AI assistance is permitted. Hidden AI assistance is not. Students receive credit for useful prompts, visible decision-making, good Git evidence, accurate reflection, and verified Java understanding.

2 Coursework Overview

2.1 Project Title

AI-Assisted Information Management System for Honor of Kings

2.2 Task Summary

Students must design and implement a Java system that manages players, heroes, equipment, teams, and match records for *Honor of Kings*. The system should support searching, reporting, data management, authentication, leaderboards, and optional advanced features.

Students are expected to use AI tools in a controlled and documented way. The final submission must show:

- a working Java application;
- a clear object-oriented design;
- a detailed `plan.md`;
- actual AI prompts and multi-agent interaction records;
- Git history showing iterative development;
- testing evidence and final reflection.

2.3 Learning Outcomes

After completing this coursework, students should be able to:

1. design a Java program using classes, inheritance, interfaces, collections, and encapsulation;
2. implement a menu-driven application with search, ranking, data management, and authentication features;
3. save and load structured data using file I/O or another persistence method;
4. use Git to document an iterative development process;
5. use AI tools responsibly by recording prompts, evaluating outputs, and explaining decisions;
6. test and debug AI-assisted code rather than trusting generated code blindly.

3 Core Java Design Requirements

3.1 Required Classes

The project must include at least the following classes:

Class	Required purpose
Person	Abstract superclass for system users.
Player	Subclass of Person; represents a game player.
Admin	Subclass of Person; represents a user with data-management permission.
Hero	Represents a playable hero, including type, stats, and equipment compatibility.
Equipment	Represents an item that can be equipped by heroes.
Team	Represents a team containing multiple players.
MatchRecord	Represents a match result, participants, hero picks, and date.

Students may add extra classes where useful, for example:

```
Skin, Rune, Skill, GameDataManager, AuthenticationService,  
SearchService, RankingService, FileStorageService,  
RecommendationEngine, CombatSimulator, InputHelper
```

3.2 Required Java Concepts

The implementation must demonstrate meaningful use of the following concepts:

Concept	Expected evidence
Inheritance	Player and Admin extend Person.
Association	A Player owns multiple Hero objects; a Hero can use multiple Equipment objects.

Aggregation composition	or A Team contains multiple Player objects.
Interface	Use at least one meaningful interface such as Searchable, Reportable, Persistable, or Authenticatable.
Encapsulation	Fields should be private; access should be controlled through appropriate methods.
Polymorphism	Use common superclass or interface references where appropriate, such as storing users as Person.
Collections	Use ArrayList, HashMap, Set, TreeMap, or similar collections.
Exception handling	Handle invalid input, missing records, duplicate IDs, and file loading errors.
File I/O	Save and load system data from text, CSV, JSON, or another documented format.
Enums	Use enums such as HeroType, MatchResult, Role, or EquipmentType.

Important

A program that places almost all logic in one large Main class will receive a low Java design score, even if some menu functions work.

4 Initial Dataset Requirements

The initial dataset must include at least:

Data type	Minimum requirement
Teams	3 teams, each with at least 5 players.
Players	10 players, each owning at least 3 heroes.
Heroes	15 heroes, each able to equip at least 2 equipment items.
Equipment	20 equipment items.
Match records	10 match records.

The dataset may be hard-coded during early development. For full marks, the system should support saving and loading data through files.

5 Functional Requirements

The system must provide a menu-driven interface. A console interface is acceptable. A GUI using Swing or JavaFX may be submitted as an extra-credit feature.

5.1 Player Lookup

Search a player by ID or name. Display:

- player ID and name;
- team;

- level and win rate;
- owned heroes;
- each hero's equipped items.

5.2 Team Overview

Search a team by ID or name. Display:

- team name;
- all members;
- average level;
- total matches;
- win rate;
- top player in the team.

5.3 Hero Details

Search a hero by name. Display:

- hero name and hero type;
- base stats;
- available or compatible equipment;
- players who own this hero;
- recommended equipment if implemented.

5.4 Equipment Statistics

Rank equipment by at least one metric, such as:

- usage count;
- average rating;
- number of heroes using it;
- win-rate contribution;
- a custom score.

Students must explain the ranking formula in the documentation.

5.5 Match History

Retrieve the last N matches for a player or team. Display:

- opponent;
- date;
- result;
- heroes picked;

- win/loss record;
- hero pick rate.

5.6 Leaderboard

Display top X players by:

- win rate;
- level;
- number of matches;
- custom score.

Students must explain how ties are handled.

5.7 Data Management

Admin users can add, delete, and edit:

- players;
- heroes;
- equipment;
- teams;
- match records.

Player users can:

- view their own information;
- edit limited personal information;
- view their heroes and match history;
- view public hero, team, and leaderboard information.

5.8 Authentication

Implement a simple login and logout system with at least two roles:

- Admin;
- Player.

Admin users can create, modify, or delete all data. Player users can only view general data and edit their own basic information.

6 Required AI Usage and Evidence

AI tools are allowed and expected. Students may use tools such as ChatGPT, Claude, Gemini, Copilot, Cursor, Codex, or other AI coding assistants. However, students are responsible for every line of submitted code.

Submitting AI-generated code that the student cannot explain will receive a low score even if the program runs.

6.1 Required AI Folder

Submit an `ai/` folder with the following files:

```
ai/  
  prompts.md  
  agent-log.md  
  reflection.md
```

6.2 Prompt Record: `prompts.md`

Students must record all important prompts used during the project. Each prompt record must include:

- date and time;
- AI tool or model used;
- agent role;
- the actual prompt;
- short summary of the AI response;
- whether the response was accepted, modified, or rejected;
- related Git commit hash.

Listing 1: Example prompt record

```
## Prompt 03  
  
Time: 2026-05-12 21:30  
Tool/Model: ChatGPT / GPT-5.5  
Agent Role: Java Architect  
Related Commit: a13f91c  
  
### My Prompt  
  
I am designing a Java OOP coursework project for an Honor of Kings  
information system. Please suggest a class structure using inheritance,  
interfaces, and collections. Do not write full code yet. Focus on class  
responsibility and relationships.  
  
### AI Response Summary  
  
The AI suggested Person, Player, Admin, Hero, Equipment, Team, and  
MatchRecord. It also suggested interfaces such as Searchable and Persistable.  
  
### My Decision  
  
Accepted the general structure but changed Match to MatchRecord to avoid  
ambiguous naming and to make the class responsibility clearer.
```

Required

Writing only “I used ChatGPT” is not enough. Students must show the actual prompts and explain what they did with the AI response.

6.3 Multi-Agent Evidence: agent-log.md

Students must use at least three different AI agent roles during development. These do not have to be different AI tools; they can be different roles assigned to the same model.

Required roles:

- **Architect Agent:** class design, UML suggestions, module planning;
- **Implementation Agent:** selected Java methods or class implementation;
- **Testing/Reviewer Agent:** bug finding, test cases, code review.

Optional roles include Refactoring Agent, Documentation Agent, Security Agent, and Performance Agent.

Listing 2: Example agent-log.md entry

```
# Agent Log

## Architect Agent

Main contribution:
Suggested the initial OOP structure and relationships.

Human decision:
I accepted the abstract Person class but rejected the suggested GameAccount
class because it was unnecessary for the required functions.

Related commits:
- 1a2b3c4 initial class design
- 2b3c4d5 add Person, Player, Admin

## Testing Agent

Main contribution:
Found that deleting a Hero did not remove it from Player-owned hero lists.

Human decision:
I fixed the bug manually by updating deleteHero in GameDataManager.

Related commits:
- 9f8e7d6 fix hero deletion consistency
```

6.4 Reflection: reflection.md

Students must answer the following questions:

1. Which AI tools or models did you use?
2. Which prompt was the most useful? Why?
3. Which AI-generated suggestion was wrong, incomplete, or misleading?
4. How did you check whether AI-generated code was correct?
5. What bugs did you fix yourself instead of asking AI to fix?
6. What Java concept did you understand better after using AI?
7. What Java concept are you still unsure about?

8. Did AI make the project easier, harder, or both? Explain.
9. Which parts of the final project were mainly written by you?
10. Which parts were mainly generated or heavily assisted by AI?

7 Required Git Evidence

Students must use Git throughout the project. The submitted repository must include meaningful commit history. A single final commit is not acceptable.

7.1 Minimum Git Requirements

- At least 12 meaningful commits.
- At least 4 commits must be clearly human-authored planning, debugging, or refactoring commits.
- At least 3 commits must be linked to Architect Agent.
- At least 3 commits must be linked to Implementation Agent.
- At least 2 commits must be linked to Testing/Reviewer Agent.

Each commit message should use one of the following prefixes:

```
[Human]
[AI-Architect]
[AI-Implementation]
[AI-Review]
[AI-Refactor]
[Docs]
[Test]
[Fix]
```

Example commit messages:

```
[Human] create initial project structure and README
[AI-Architect] draft OOP class structure for Person Hero Team
[Human] revise class design based on UML feedback
[AI-Implementation] implement player lookup menu
[Test] add manual test cases for team overview
[AI-Review] review data deletion consistency
[Fix] fix hero deletion bug found during review
[Docs] update plan.md and reflection.md
```

Students must include the output of:

```
git log --oneline --graph --decorate --all
```

in a file named git-history.txt.

Marker Note

Git evidence is assessed for development process, not just quantity. A long list of meaningless commits is weaker than a shorter but clear sequence of planning, implementation, review, fixes, and documentation.

8 Required plan.md

Students must submit a carefully written `plan.md`. This file is a major part of the coursework.

Required structure for plan.md

1. **Project Goal:** explain what the system does and who the users are.
2. **Requirement Analysis:** list all required features and explain how the system will implement each one.
3. **Java Concepts Used:** explain where inheritance, interfaces, polymorphism, collections, exception handling, file I/O, and enums will be used.
4. **Class Design:** describe each major class and its responsibility.
5. **UML Draft:** include a UML diagram image or text-based UML description.
6. **Data Design:** explain the initial dataset and how data is stored.
7. **AI Usage Plan:** explain which AI agents will be used and what each agent is allowed to help with.
8. **Prompt Strategy:** explain how prompts will be written and how AI outputs will be checked.
9. **Development Timeline:** break the project into stages.
10. **Testing Plan:** list test cases for each major function.
11. **Risk Analysis:** explain possible risks and mitigation strategies.
12. **Final Reflection Placeholder:** leave space for final reflection after implementation.

A weak `plan.md` that only says “I will use AI to help me code” will receive low marks. A strong `plan.md` should show that the student planned the project before coding.

8.1 Suggested Timeline

Stage	Expected work
Stage 1	Read requirements, create repository, write first <code>plan.md</code> .
Stage 2	Ask Architect Agent for design feedback; revise class structure manually.
Stage 3	Implement model classes and initial data.
Stage 4	Implement menu system and search features.
Stage 5	Implement authentication and admin/player permissions.
Stage 6	Implement persistence and ranking functions.
Stage 7	Use Testing/Reviewer Agent to find bugs; fix and record decisions.
Stage 8	Finish documentation, reflection, Git export, and final testing.

9 Source Code and Testing Requirements

9.1 Source Code Requirements

The Java project must:

- compile successfully;
- run from a clear main method;
- have a clear package or folder structure;
- use meaningful class, method, and variable names;
- avoid putting all code into one giant class;
- avoid unnecessary duplicated code;
- handle invalid user input gracefully;
- include comments for important logic;
- be understandable to a human marker.

Suggested structure:

```
src/  
  Main.java  
model/  
  Person.java  
  Player.java  
  Admin.java  
  Hero.java  
  Equipment.java  
  Team.java  
  MatchRecord.java  
service/  
  GameDataManager.java  
  AuthenticationService.java  
  SearchService.java  
  RankingService.java  
  FileStorageService.java  
util/  
  InputHelper.java  
  DataInitializer.java  
docs/  
  plan.md  
  design.md  
 uml.png  
  test-cases.md  
ai/  
  prompts.md  
  agent-log.md  
  reflection.md  
git-history.txt  
README.md
```

9.2 Testing Requirements

Students must include either automated tests or manual test documentation. At minimum, submit:

```
docs/test-cases.md
```

The testing document must include at least 10 test cases. Each test case should include:

- test ID;

- function tested;
- input;
- expected output;
- actual output;
- pass/fail result;
- bug found, if any.

Listing 3: Example test case

```
## Test 04: Player Lookup by Name

Input:
Search player name: "Li Bai"

Expected:
The system displays Li Bai's team, level, owned heroes, and equipment.

Actual:
The system displayed player information correctly.

Result:
Pass
```

Bonus credit may be awarded for meaningful JUnit tests.

10 Extra Credit Features

Students may implement one or more of the following features for extra credit.

10.1 Combat Simulation

Implement a turn-based battle simulator using hero stats, equipment, and random factors. It should include damage calculation, critical hit or dodge, win/loss result, and a combat report.

10.2 Recommendation Engine

Recommend heroes or equipment based on hero type, player preference, win rate, equipment usage, or team composition. The formula must be explained in the documentation.

10.3 GUI

Implement a GUI using Swing or JavaFX. It should support at least player lookup, team overview, hero details, and leaderboard.

10.4 Data Persistence

Save and load data using text files, CSV files, JSON files, or JDBC database. External libraries are allowed only if clearly documented.

10.5 Advanced AI Reflection

Compare two different AI models or two different agent roles solving the same problem. For example, ask both an Implementation Agent and a Reviewer Agent how to implement leaderboard sorting, then compare correctness, readability, bugs, and learning value.

11 Submission Requirements

Submit a .zip file or a Git repository link containing:

```
src/  
docs/  
ai/  
README.md  
plan.md  
git-history.txt
```

The submission must include:

- complete Java source code;
- plan.md;
- design document or design.md;
- UML class diagram;
- prompts.md;
- agent-log.md;
- reflection.md;
- git-history.txt;
- testing document;
- instructions for running the program.

Do not submit only screenshots. Do not submit only AI chat logs without source code. Do not submit code that you cannot explain.

12 Academic Integrity and AI Policy

AI use is allowed in this coursework. However, the following are not allowed:

- submitting AI-generated code without understanding it;
- deleting or hiding AI prompts;
- faking Git history;
- claiming AI-generated work as entirely your own;
- using AI to fabricate test results;
- using AI to write the reflection dishonestly;
- submitting another student's code.

Students may use AI for:

- understanding requirements;
- brainstorming design;
- generating small code examples;
- debugging;
- refactoring;
- writing test cases;
- improving documentation.

Students must personally verify that:

- the code compiles;
- the code runs;
- the code matches the requirements;
- Java concepts are used correctly;
- the documentation accurately describes the final system.

13 Grading Rubric

Category	Marks	Criteria
Java Design and Understanding	5	Excellent submissions show clear OOP design, inheritance, interfaces, collections, encapsulation, exception handling, file I/O, and explainable design decisions. Weak submissions are mostly procedural or cannot be explained.
Functional Completeness	4	Full marks require working lookup, team overview, hero details, equipment statistics, match history, leaderboard, data management, and authentication.
AI Usage Evidence	4	Full marks require actual prompt records, multi-agent workflow, strong reflection, and clear explanation of accepted, modified, and rejected AI suggestions.
Git Process Evidence	3	Full marks require clear iterative Git history showing human planning, AI-assisted implementation, review, fixes, tests, and documentation.
plan.md and Documentation	2	Full marks require detailed planning, UML, design explanation, testing plan, implementation notes, and accurate final documentation.
Testing and Reliability	1	Full marks require clear manual test cases or automated tests with evidence of debugging.
Extra Credit or Creativity	1	Award for meaningful extra feature such as GUI, persistence, recommendation engine, combat simulation, or advanced AI reflection.

13.1 Grade Bands

Grade	Marks	Description
A	16-20	Core features work, Java design is strong, AI and Git evidence are detailed, documentation is professional, and at least one extra feature is implemented.
B	10-15	Most core features work, Java structure is acceptable, AI evidence and documentation are present but not excellent.
C	6-9	Some required features are incomplete, Java design is shallow, documentation or AI evidence is weak.
F	0-5	Major functionality is missing, code does not run, work is disorganized, AI use is hidden, or documentation is inadequate.

14 Minimum Passing Checklist

Before submission, students should confirm that:

- the program runs;
- there are at least 7 required classes;
- there are at least 10 players;
- there are at least 15 heroes;
- there are at least 20 equipment items;
- there are at least 3 teams;
- there are at least 10 match records;
- the menu system works;
- player lookup works;
- team overview works;
- hero details work;
- equipment statistics work;
- leaderboard works;
- admin/player login works;
- `plan.md` exists and is detailed;
- `prompts.md` contains actual prompts;
- `agent-log.md` shows at least 3 agent roles;
- `reflection.md` answers all questions;
- Git has at least 12 meaningful commits;
- `git-history.txt` is included;
- testing document is included.

Appendix A: Student Workflow

1. Read the coursework carefully.
2. Create a Git repository.
3. Write `plan.md` before coding.
4. Ask Architect Agent for design feedback.
5. Create initial Java classes manually.
6. Commit the initial structure.
7. Use Implementation Agent only for selected methods, not the entire project at once.
8. Compile and run frequently.
9. Use Testing/Reviewer Agent to review code.
10. Fix bugs and record what happened.
11. Update `prompts.md`, `agent-log.md`, and `reflection.md`.
12. Export Git history.
13. Final check: make sure every class and method can be explained.

Appendix B: Prompt Quality Guide

Quality	Weak prompt	Stronger prompt
Design	"Write my Java project."	"Suggest a class design for a Java OOP Honor of Kings information system. Include inheritance, interfaces, collections, and class responsibilities. Do not write full code."
Implementation	"Code this."	"Implement only the player lookup method using the existing Player, Hero, and Team classes below. Explain assumptions and edge cases."
Debugging	"Fix my code."	"The following method crashes when searching an unknown hero. Identify the cause, explain it, and suggest a minimal fix. Do not rewrite unrelated code."
Review	"Is this good?"	"Review this Java class for OOP design, encapsulation, collection usage, and potential null pointer bugs. Give specific comments."

Appendix C: Recommended README.md Structure

```
# AI-Assisted Honor of Kings Information Management System

## 1. Project Overview
```

```
## 2. How to Run
## 3. Default Login Accounts
## 4. Implemented Features
## 5. Java Concepts Used
## 6. AI Usage Summary
## 7. Testing Summary
## 8. Known Limitations
```

References

1. Stanford CS106B Archive. *Using AI to Help Code*. <https://web.stanford.edu/class/archive/cs/cs106b/cs106b.1262/assignments/7-huffman-ai-coding/>
2. Stanford Teaching Commons. *Creating your course policy on AI*. <https://teachingcommons.stanford.edu/teaching-guides/artificial-intelligence-teaching-guide/creating-your-course-policy-ai>
3. Stanford SCALE. *Promptly: Using Prompt Problems to Teach Learners How to Effectively Utilize AI for Code Generation*. <https://scale.stanford.edu/ai/repository/promptly-using-prompt-problems-teach-learners-how-effectively-utilize-ai-code>